Arquitetura geral — RISC-V, DMA e malha sistólica

Entrega Parcial I | Integração proposta | C = A × B

irq_done: conexão futura; ainda ausente na CPU

Memória de instruções

IMEM: 1 KB, palavras de 32 bits
Espaço separado da DMEM

PC [31:0]

instrução [31:0]

CPU RISC-V RV32I

Sem pipeline; espera valid/ready
Configura a operação por MMIO

Interface de dados
ver quadro 1

Ponte CPU–AXI4

Mestre 0: uma palavra por transação
Converte requisição e resposta

AXI4 • mestre 0

Controlador DMA

Registradores MMIO: 0x40000000–0x4000002F
Bases A/B/C, dimensões M/K/N, START e status
Controla tiling (ti, tj, tk) e zero-padding
Mestre AXI4: lê A/B e grava C

AXI4 • mestre 1 (DMA)

AXI4 • escravo MMIO

Interconexão AXI4

Árbitro + decodificador de endereços
Prioridade ao DMA; sem interromper transação
Canais AW, W, B, AR e R; dados de 32 bits

leituras / escritas

Memória de dados compartilhada

DMEM: 16 KB • escravo AXI4
0x00000000–0x00003FFF
Bases padrão: A=0x1000, B=0x2000,
C=0x3000; um elemento/palavra

DMA → acelerador
escrita A/B + seleção de C
acc_clear / acc_start
ver quadro 2

Acelerador → DMA
acc_busy / acc_done
buf_c_rdata [31:0]

Acelerador matricial

Buffers locais A/B

Cada buffer: T × T int8
Recebem dados e índices
do DMA

int8

Skew

Atrasos por linha
e por coluna
T entradas A/B

int8

Malha sistólica T × T

T = 2, 4 ou 8 • MACs
A/B: int8; acumuladores: int32
C permanece nos PEs

Fluxo da operação

1. CPU configura e inicia o DMA.
2. DMA lê A/B e carrega os tiles.
3. Malha acumula os produtos.
4. DMA grava C e sinaliza DONE.

1. Interface CPU ↔ ponte

CPU → ponte: dmem_valid, dmem_write,
dmem_addr[31:0], dmem_wdata[31:0], dmem_wstrb[3:0]
Ponte → CPU: dmem_ready, dmem_rdata[31:0], dmem_error
Conclusão na borda com valid && ready.
A CPU mantém a requisição estável durante a espera.

2. Interface DMA ↔ acelerador

DMA → acelerador: buf_a/b_we, buf_a/b_row, buf_a/b_col,
buf_a/b_wdata[7:0], buf_c_row, buf_c_col, acc_clear, acc_start
Acelerador → DMA: acc_busy, acc_done, buf_c_rdata[31:0]
Índices de linha/coluna: clog2(T) bits.
Escrita nos buffers somente com acelerador ocioso.

Comuns aos blocos: clk e rst_n (reset ativo em zero). Setas duplas representam requisições e respostas.

A IMEM já está dentro de rv32i_core; aparece separada para mostrar a arquitetura Harvard.

Diagrama da integração planejada. Os detalhes e o mapa completo de registradores estão na especificação.